OPERATING SYSTEM

ORANGE PROBLEM - 2

NAME : A VINAY

SRN:PES1UG24CS647

Phase 1: Object Storage Foundation

1a:

```
vinay@vinay-VirtualBox:~$ cd PES1UG24CS647-pes-vcs
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ make test_objects
make: 'test_objects' is up to date.
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$
```

1b:

```
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$
```

Phase 2: Tree Objects

2a:

```
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$
```

2b:

```
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ ./pes commit -m "test"
Committed: 6e6ea38c221e... test
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ find .pes/objects -type f
.pes/objects/dd/7493dd11a4e1855394b78721d8c44a16ff6db5efb484e209c638552d65d941
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/6e/6ea38c221e6618a73159ad8ccdd6892b3c52cd4360bdbf11bc820bcc54fefb
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ hexdump -C $(find .pes/objects -type f | head -n 1)
00000000  74 72 65 65 20 34 39 00  31 30 30 36 34 34 20 64  |tree 49.100644 d|
00000010  75 6d 6d 79 2e 74 78 74  00 00 00 00 00 00 00 00  |ummy.txt........|
00000020  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
00000030  00 00 00 00 00 00 00 00  00                       |.........|
00000039
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$
```

Phase 3:Index

3a:

```
00000039
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ ./pes init
echo "hello" > file.txt
./pes add file.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
  staged:     a.txt
  staged:     file.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.h
  untracked:  tree.h
  untracked:  .gitignore
  untracked:  tree.c
  untracked:  Makefile
  untracked:  README.md
  untracked:  test_objects.c
  untracked:  test_tree
  untracked:  index.c
  untracked:  object.c
  untracked:  test_sequence.sh
  untracked:  pes.h
  untracked:  commit.c
  untracked:  test.txt
  untracked:  text.txt
  untracked:  index.h
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  test_objects

vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$
```

3b:

```
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776321297 6 a.txt
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776321607 6 file.txt
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$
```

Phase 4: Commits and History

4a:

```
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776321607 6 file.txt
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ ./pes commit -m "first commit"
Committed: bdf8d8d76fc2... first commit
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$
```

```
Committed: bdf8d8d76fc2... first commit
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ ./pes log
commit bdf8d8d76fc20e69a941da9d67c4dafb9fbdbe9016b762a3e0dbcff274950822
Author: PES User <pes@localhost>
Date:   1776321701

    first commit

commit 6e6ea38c221e6618a73159ad8ccdd6892b3c52cd4360bdbf11bc820bcc54fefb
Author: PES User <pes@localhost>
Date:   1776321423

    test

vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$
```

4b:

```
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$
```

4c:

```
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$
```

```
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$ cat .pes/refs/heads/main
bdf8d8d76fc20e69a941da9d67c4dafb9fbdbe9016b762a3e0dbcff274950822
vinay@vinay-VirtualBox:~/PES1UG24CS647-pes-vcs$
```

Questions and report:

Q5.1: Branching and Checkout

In systems like Git or PES, a branch is essentially represented as a file inside .pes/refs/heads/ that contains the hash of a specific commit. Implementing the pes checkout <branch> command involves several coordinated steps:

1. First, the .pes/HEAD file needs to be modified so that it refers to the chosen branch. This is done by setting it to ref: refs/heads/<branch>.

2. Next, the commit hash stored in the corresponding branch file (.pes/refs/heads/<branch>) must be retrieved.

3. Using this hash, the related commit object is loaded, and its associated tree structure is extracted.

4. The working directory is then synchronized with this tree. This process may involve removing outdated tracked files, updating modified ones, and recreating files based on blob objects stored in the system.

5. Finally, the index (staging area) must also be aligned with the updated tree so that it reflects the current state accurately.

This entire process is non-trivial because it requires careful handling of file changes, directory structures, and ensuring that any ongoing (uncommitted) work is not accidentally lost.

Q5.2: Detecting Dirty Working Directory Conflicts

When switching branches, the system must ensure that no important changes are overwritten. To do this, it compares three versions of each tracked file: the current working directory, the index, and the version in the target branch.

1. Initially, the system checks whether the file in the working directory has been altered compared to the index. This can be done using attributes like file size or last modification time.

2. If a difference is detected, the next step is to compare the indexed version of the file with the version present in the branch being checked out.

3. If both comparisons show differences, it means there is a conflict situation.

A conflict arises when:

- The working directory contains changes that are not yet committed, and
- The same file also differs between the index and the target branch.

In such cases, the checkout process must be stopped to avoid losing the user's uncommitted work.

Q5.3: Detached HEAD

A detached HEAD state happens when the .pes/HEAD file directly stores a commit hash instead of pointing to a branch reference.

In this condition, it is still possible to create new commits. However, these commits are not linked to any branch, which makes them difficult to access later. If the user switches to another branch, such commits may become unreachable and could eventually be deleted during garbage collection.

To prevent losing this work, a new branch can be created that points to the current commit. This can be done using commands like:

- git branch <new_branch> <commit_hash>
- or git checkout -b <new_branch>

By doing this, the commits are attached to a branch again and remain safely accessible

Q6.1: Garbage Collection and Reachability

As time passes, the object storage may contain many objects that are no longer referenced by any branch. These unused objects need to be removed to save space, which is done using a mark-and-sweep approach.

1. The process begins by identifying all branch heads located in .pes/refs/heads/.
2. From each branch head, the system traverses through all linked commits and recursively visits related tree and blob objects.
3. Every object encountered during this traversal is marked as reachable and stored in a collection (such as a hash set).
4. After marking is complete, the system scans all objects present in .pes/objects/.

5. Any object that is not found in the reachable set is considered unused and is deleted.

A hash set is preferred for storing reachable objects because it allows fast lookups, which is important when dealing with large repositories.

For example, in a repository with around 100,000 commits and multiple branches, the total number of objects (including commits, trees, and blobs) can reach several hundred thousand. Efficient tracking is therefore essential.

Q6.2: Garbage Collection Race Condition

Running garbage collection at the same time as a commit operation can lead to serious issues due to race conditions.

Consider this scenario:

A commit process creates new objects (such as blobs, trees, and a commit), but the branch reference has not yet been updated.

Meanwhile, garbage collection scans the object storage.

Since the new objects are not yet linked to any branch, they are treated as unreachable and removed.

Later, when the commit process tries to update the branch to point to these objects, it results in inconsistencies or corruption.

To avoid such problems, systems like Git use several safeguards:

Locking mechanisms to prevent simultaneous conflicting operations

Atomic updates to ensure references are changed only after all objects are safely written

Temporary references to protect newly created objects

A delay (grace period) before deleting unused objects

These techniques ensure that important data is not accidentally removed and the repository remains consistent.